NAME ROHAN RESHI

PRN:202401040034

BATCH:A1-6

PEC AIML LAB ASSIGNMENT NO. 1

Smart Office Vacuum Cleaner Using Python

Aim

To design and implement a simple smart office vacuum cleaner agent in Python that can identify dirty areas in an office, clean them automatically, collect the dirt in a dustbin, and monitor its battery health during the cleaning operation.

Experiment Objective and Problem Statement

Objective

The main objective of this experiment is to understand how an intelligent agent works in a real-life office environment and how it can take suitable actions based on the current condition of the environment.

In this experiment, a smart vacuum cleaner is designed using Python. The environment represents an office containing four different areas:

- Reception
- Cabin
- Meeting Room
- Work Area

Each area can be either Clean or Dirty.

The vacuum agent checks each area and decides whether it needs cleaning. When a dirty room is found, the vacuum cleans it, collects the dirt in its dustbin, and reduces its battery level.

Problem Statement

A normal vacuum cleaner requires a person to control its cleaning operation. A smart vacuum cleaner should be able to identify the condition of an area and take an appropriate action automatically.

The problem is to develop a simple Python-based smart office vacuum cleaner agent that can:

1. Represent an office environment with different areas.
2. Identify whether an office area is Clean or Dirty.
3. Move through different office areas.
4. Clean an area when dirt is detected.
5. Leave an already clean area unchanged.
6. Collect the dirt removed from dirty areas.
7. Monitor the battery health of the vacuum.
8. Reduce the battery level when cleaning is performed.
9. Display the final condition of the office.
10. Display the final battery level and amount of collected dirt.

The system uses simple rule-based decision making.

Experiment Outcomes

After completing this experiment, the following outcomes were achieved:

1. A basic intelligent vacuum cleaner agent was implemented using Python.
2. A real-life office environment was created.
3. The environment contains four areas: Reception, Cabin, Meeting Room, and Work Area.
4. Each office area has a condition of either Clean or Dirty.
5. The vacuum agent checks the condition of each office area.
6. The vacuum automatically cleans an area when it is Dirty.
7. The vacuum does not clean an area that is already Clean.
8. The collected dirt is stored in the vacuum's dustbin.
9. The battery level decreases whenever the vacuum cleans a dirty area.
10. The final room status, battery level, and collected dirt are displayed.

Conceptual Details / Implementation

1. Environment

The environment represents a real-life office containing four different areas:

- Reception
- Cabin
- Meeting Room
- Work Area

Each area is assigned one of two conditions:

- Clean → The area does not require cleaning.
- Dirty → The area requires cleaning.

The office environment is represented using a 2D list.

The initial environment is:

Reception → Dirty
Cabin → Clean
Meeting Room → Dirty
Work Area → Dirty

The `OfficeEnvironment` class is used to create and manage the office environment.

---

2. Vacuum Cleaner Agent

The `VacuumAgent` class represents the smart vacuum cleaner.

The vacuum agent is connected to the office environment using:

`self.environment = environment`

This allows the vacuum agent to access the rooms stored inside the office environment.

The main function of the vacuum agent is:

`clean_office()`

This function makes the vacuum visit every office area and perform the required action according to its condition.

The agent also has two important properties:

`self.battery = 100`

`self.dustbin = 0`

Battery

The battery starts at 100%.

Whenever the vacuum cleans a dirty room, its battery decreases by 10%.

Dustbin

The dustbin starts at 0.

Whenever the vacuum cleans a dirty room, one unit of collected dirt is added to the dustbin.

---

3. Room Status

The condition of each office area is stored together with its name.

Example:

`["Reception", "Dirty"]`

Here:

- `room[0]` represents the room name.
- `room[1]` represents the room condition.

The vacuum checks `room[1]` to determine whether cleaning is required.

---

4. Room Status Display

The `show_rooms()` function is used to display the current condition of all office areas.

The function uses a loop:

`for room in self.rooms:`

The room name and its condition are displayed using:

`print(room[0], ":", room[1])`

This allows the user to see the condition of the office before and after the cleaning operation.

5. Dirt Detection

The vacuum agent checks the current condition of each office area.

If the condition is:

Dirty

the vacuum identifies that cleaning is required.

If the condition is:

Clean

the vacuum identifies that no cleaning action is required.

The decision is made using a simple if-else condition.

---

6. Cleaning Action

When the vacuum finds a dirty area, it performs the cleaning action.

The condition is changed using:

`room[1] = "Clean"`

Example:

Before cleaning:

Reception → Dirty

After cleaning:

Reception → Clean

Therefore, the agent changes the state of the environment after performing the cleaning action.

---

7. Battery Health

The vacuum cleaner has a battery level that starts at:

`self.battery = 100`

The battery represents the available energy of the vacuum.

Whenever the vacuum cleans a dirty room, the battery decreases by 10%:

`self.battery = self.battery - 10`

Example:

Initial Battery = 100%

Clean Reception → Battery = 90%

Clean Meeting Room → Battery = 80%

Clean Work Area → Battery = 70%

The Cabin is already clean, so no battery is used for cleaning it.

Therefore:

Final Battery = 70%

---

8. Dirt Collection / Dustbin

The vacuum cleaner has a simple dustbin for storing collected dirt.

The dustbin starts empty:

`self.dustbin = 0`

Whenever a dirty room is cleaned, one unit of dirt is added:

`self.dustbin = self.dustbin + 1`

Initially:

Dustbin = 0

After cleaning Reception:

Dustbin = 1

After cleaning Meeting Room:

Dustbin = 2

After cleaning Work Area:

Dustbin = 3

Therefore:

Final Dirt Collected = 3 units

---

9. Movement Through Office Areas

The vacuum visits the office areas one by one using:

`for room in self.environment.rooms:`

The vacuum checks the following areas:

1. Reception
2. Cabin
3. Meeting Room
4. Work Area

After checking one area, it moves to the next area.

The `time.sleep(1)` function is used to create a one-second delay between the operations, making the simulation more realistic.

10. Decision Making

The vacuum uses a simple rule-based decision-making process.

The basic rules are:

IF room is Dirty
THEN Clean the room
AND Collect Dirt
AND Reduce Battery

IF room is Clean
THEN Do Nothing

The agent observes the current condition of the room and performs the appropriate action.

Agent and Environment Interaction

The office environment provides the current condition of each area to the vacuum agent.

The vacuum agent observes the condition and performs an action.

The interaction can be represented as:

OFFICE ENVIRONMENT ↓ Current Room Status ↓ VACUUM AGENT ↓ Observe Room ↓ Is Room Dirty? /
YES NO ↓ ↓ Clean Room No Action ↓ Collect Dirt ↓ Reduce Battery ↓ Next Room ↓ Repeat Process ↓ All Rooms Checked ↓ Display Final Status ↓ END

Simple Reflex Agent

The Smart Office Vacuum Cleaner can be considered a Simple Reflex Agent.

A simple reflex agent selects an action based on the current condition of its environment.

The rules for this vacuum are:

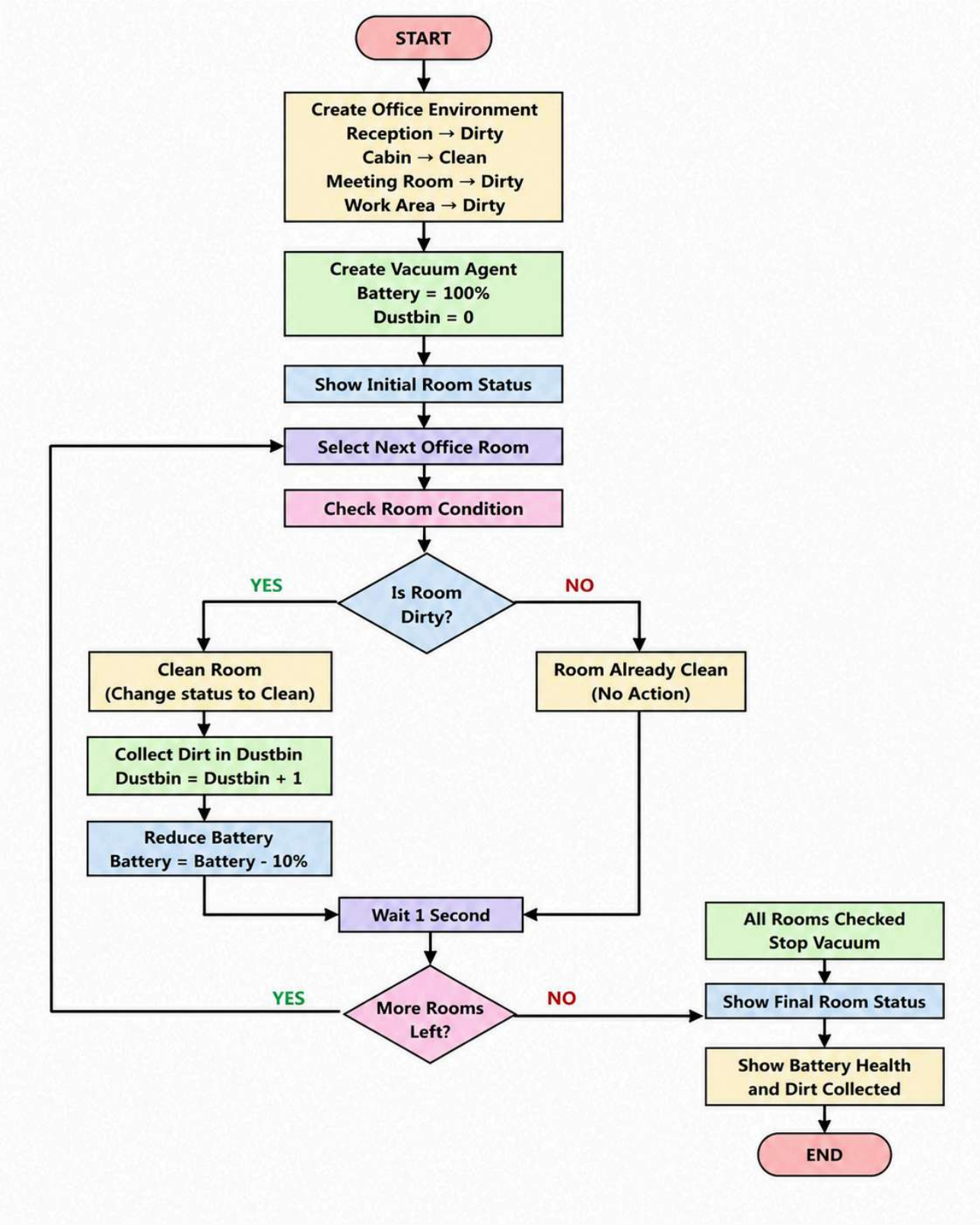
IF room is Dirty
THEN Clean Room
AND Collect Dirt
AND Reduce Battery

IF room is Clean

THEN Do Nothing

The agent observes the current room and immediately performs the appropriate action.

FLOWCHART



PROGRAM CODE

import time

Environment class

class OfficeEnvironment:

```
def __init__(self):

    # Office rooms and their conditions
    self.rooms = [
        ["Reception", "Dirty"],
        ["Cabin", "Clean"],
        ["Meeting Room", "Dirty"],
        ["Work Area", "Dirty"]
    ]

# Display the current status of all rooms
def show_rooms(self):

    print("\nOffice Room Status:")

    for room in self.rooms:
        print(room[0], ":", room[1])
```

Vacuum Agent class

class VacuumAgent:

```
def __init__(self, environment):

    # Connect vacuum agent with office environment
    self.environment = environment

    # Battery starts at 100%
    self.battery = 100

    # Dustbin starts empty
    self.dustbin = 0
```

```
# Function to clean the office
def clean_office(self):

    print("\nSmart Office Vacuum Cleaner Started\n")

    # Visit every room one by one
    for room in self.environment.rooms:

        room_name = room[0]
        status = room[1]

        print("Vacuum is moving to:", room_name)

        # Check whether room is dirty
        if status == "Dirty":

            print("Dirt found!")
            print("Cleaning", room_name)

            # Clean the room
            room[1] = "Clean"

            # Collect dirt in dustbin
            self.dustbin = self.dustbin + 1

            # Reduce battery by 10%
            self.battery = self.battery - 10

            print(room_name, "cleaned!")
            print("Dirt collected:", self.dustbin)
            print("Battery:", self.battery, "%\n")

        else:

            print(room_name, "is already clean.")
            print("Battery:", self.battery, "%\n")

        time.sleep(1)

    print("All office rooms have been checked.")
    print("Vacuum cleaner stopped.")

    print("\nFinal Vacuum Status:")
```

```
    print("Battery:", self.battery, "%")
    print("Dirt collected:", self.dustbin)
```

Create office environment

office = OfficeEnvironment()

Show initial room status

office.show_rooms()

Create vacuum agent

vacuum = VacuumAgent(office)

Start cleaning

vacuum.clean_office()

Show final room status

office.show_rooms()

Example Output

The program produces output similar to the following:

Office Room Status:

Reception : Dirty Cabin : Clean Meeting Room : Dirty Work Area : Dirty

Smart Office Vacuum Cleaner Started

Vacuum is moving to: Reception Dirt found! Cleaning Reception Reception cleaned! Dirt collected: 1 Battery: 90 %

Vacuum is moving to: Cabin Cabin is already clean. Battery: 90 %

Vacuum is moving to: Meeting Room Dirt found! Cleaning Meeting Room Meeting Room cleaned! Dirt collected: 2 Battery: 80 %

Vacuum is moving to: Work Area Dirt found! Cleaning Work Area Work Area cleaned! Dirt collected: 3 Battery: 70 %

All office rooms have been checked. Vacuum cleaner stopped.

Final Vacuum Status: Battery: 70 % Dirt collected: 3

Office Room Status:

Reception : Clean Cabin : Clean Meeting Room : Clean Work Area : Clean

Result Analysis

The implemented Smart Office Vacuum Cleaner successfully performed the cleaning operation according to the condition of the office environment.

Initially, the office contained three dirty areas and one clean area. The vacuum agent checked each office area one by one.

The vacuum first moved to the Reception. Since the Reception was Dirty, the vacuum detected the dirt and cleaned the area. One unit of dirt was collected and the battery decreased from 100% to 90%.

Next, the vacuum moved to the Cabin. The Cabin was already Clean, so the vacuum did not perform any cleaning action. The battery remained at 90%.

The vacuum then moved to the Meeting Room. Since the Meeting Room was Dirty, the vacuum cleaned it. A second unit of dirt was collected and the battery decreased to 80%.

Finally, the vacuum moved to the Work Area. The Work Area was Dirty, so the vacuum cleaned it. A third unit of dirt was collected and the battery decreased to 70%.

After checking all four office areas, all areas became Clean.

Initial Condition

Reception ⟶ Dirty
Cabin ⟶ Clean
Meeting Room ⟶ Dirty
Work Area ⟶ Dirty

Initial Vacuum Status

Battery ⟶ 100%
Dustbin ⟶ 0

Final Condition

Reception → Clean
Cabin → Clean
Meeting Room → Clean
Work Area → Clean

Final Vacuum Status

Battery → 70%
Dirt Collected → 3 units

The experiment demonstrates how an intelligent agent can observe the current condition of an environment, perform an appropriate action, update the environment, collect dirt, and monitor its battery health.

Conclusion

The Smart Office Vacuum Cleaner was successfully implemented using Python.

The experiment demonstrated the interaction between an intelligent agent and its environment. The office environment was represented using four areas: Reception, Cabin, Meeting Room, and Work Area.

The vacuum agent checked each area and identified whether it was Clean or Dirty. When the agent found a Dirty area, it performed the cleaning action, changed the room condition to Clean, collected the dirt in its dustbin, and reduced its battery level by 10%.

If the area was already Clean, the agent did not perform any unnecessary cleaning action.

The final result shows that all four office areas were successfully cleaned. The vacuum collected 3 units of dirt and its battery changed from 100% to 70%.

This experiment demonstrates the basic working of a Simple Reflex Agent and helps in understanding the concepts of Environment, Agent, Perception, Decision Making, Action, Battery Health, Dirt Collection, and Agent-Environment Interaction.